



JS

Fetch & Local Storage

Vito Deriu

Methode Fetch

En JavaScript la methode **fetch()** sert a faire des **requetes HTTP**. Elle permet de faire un appel a une API (c'est a dire en récupérer les données) en passant une **URL en paramètre**.

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("error :" + error);
  });
```

Methode Fetch

- On utilise l'interface **response** pour indiquer le format de la réponse. Ici du JSON.
- On affiche ensuite le **rendu de la réponse**
- On traite les **erreurs** avec **catch()** puis on affiche les erreurs

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("error :" + error);
  });
```

Methode Fetch

Les formats de **response** :

text() : retourne la réponse sous forme de chaîne de caractère

json () : retourne la réponse sous forme d'objet JSON

formData() : retourne la réponse sous forme d'objet FormData

blob() : retourne la réponse sous forme d'objet Blob



Asynchrone

Parfois les API mettent du temps à répondre. Pour que le script **continue de s'exécuter** en attendant la réponse et éviter que la page stoppe son **chargement**, on utilise une fonction **asynchrone**.

```
async function getData() {  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```




Asynchrone

On utilise le mot clé **async** pour indiquer que la fonction est asynchrone.

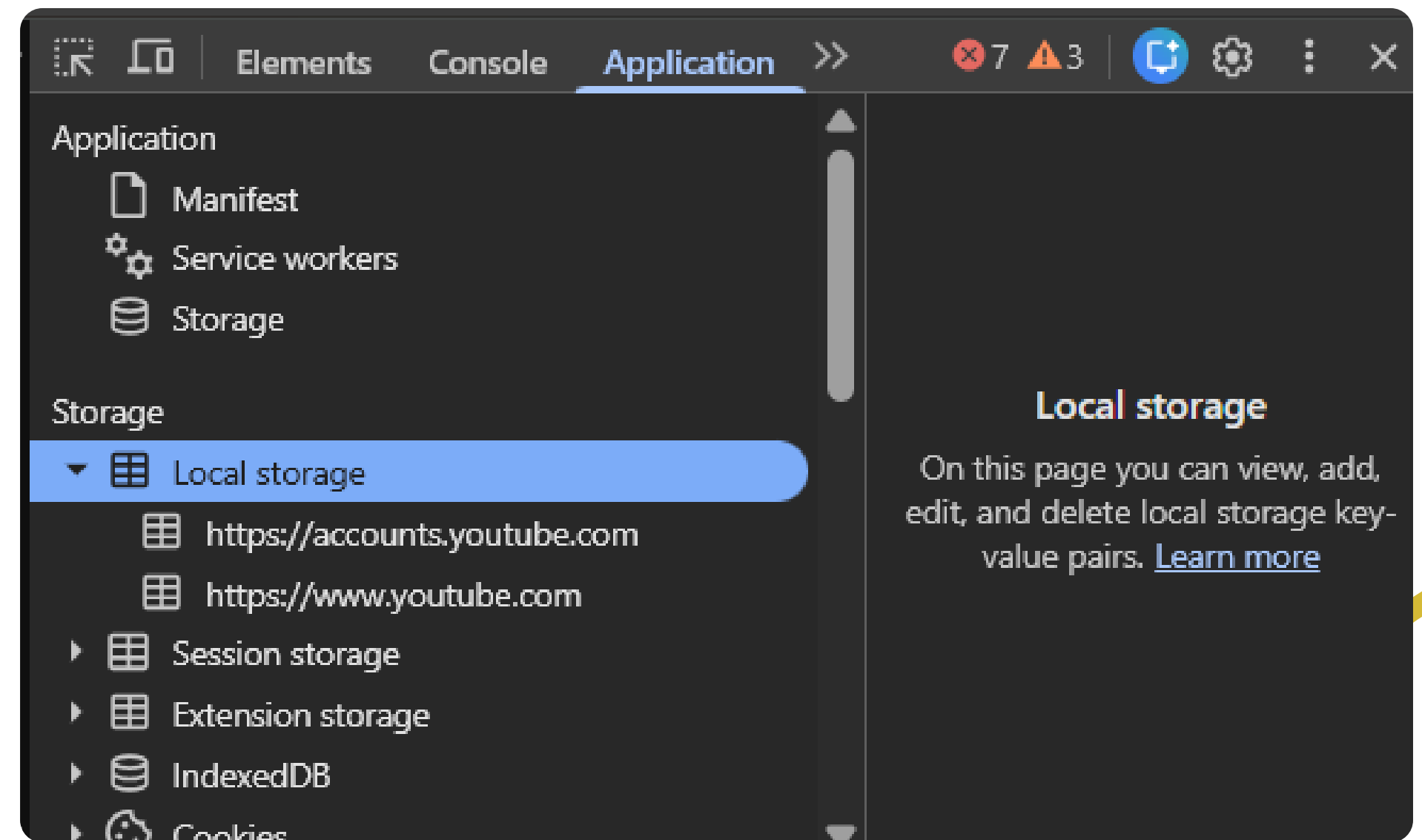
On combine avec le mot clé **await** pour indiquer que l'on attend une réponse de la part de notre appel.

```
async function getData() {  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Local Storage

Le local storage permet de **stocker** des informations dans le **navigateur** du client de manière **persistante** sous forme de chaîne de caractères en paire de **clé-valeur**

De cette façon les données restent même après un **refresh** ou après **fermeture** du navigateur et sont liées à un domaine.





Local Storage

Les methodes principales :

Ajouter un élément :

```
localStorage.setItem("ville", "Marseille")
```

Récupérer et lire un élément :

```
const ville = localStorage.getItem("ville")  
console.log(ville)
```

Supprimer un élément :

```
localStorage.removeItem("ville")
```

Tout supprimer :

```
localStorage.clear()
```


Local Storage



The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The top bar includes icons for back, forward, and search, along with tabs for 'Elements', 'Console', 'Sources', and 'Network'. Below the top bar, there are icons for a list, a filter, and a 'top' dropdown. The console log shows the following sequence of commands and their results:

```
> localStorage.setItem('ville', 'Marseille')
< undefined

> localStorage.getItem('ville')
< 'Marseille'

> localStorage.removeItem('ville')
< undefined

> |
```

The screenshot shows the Chrome DevTools Application tab. The top bar includes icons for back, forward, and search, along with tabs for 'Elements', 'Application', and '>>'. Below the top bar, there are icons for a list, a filter, and a 'file://' dropdown. The left sidebar shows the 'Application' section with a tree view containing 'Manifest', 'Service workers', and 'Storage'. Under 'Storage', there is a sub-section 'Local storage' which is expanded to show 'file://'. The right pane shows a table with the following data:

Key	Value
ville	Marseille